Лабораторная работа №5
Проектирование и создание базы данных

Дисциплина: Технологии разработки программного обеспечения
Студент: Черноусов Олег Андреевич
Группа: еозИСИП - 303
Тема: Система учёта заявок

1. Цель работы
Научиться переходить от модели предметной области к реляционной базе данных и SQL-запросам.

2. Ход работы

2.1. Сущности и связи.
На основе требований из лабораторной №1 выделены сущности: Users (пользователи), Requests (заявки), Statuses (статусы), Categories (категории), Comments (комментарии). Связи:
- Users 1:M Requests (пользователь создаёт заявки, `applicant_id`; исполнитель — `executor_id`);
- Categories 1:M Requests (`category_id`);
- Statuses 1:M Requests (`status_id`);
- Requests 1:M Comments (`request_id`);
- Users 1:M Comments (`user_id`).

2.2. ER-диаграмма.
Построена ER-диаграмма, отражающая таблицы, первичные и внешние ключи и связи 1:M. Файл: `docs/db/er_diagram.png`.

2.3. Структура таблиц.

Таблица	Первичный ключ	Обязательные поля	Внешние ключи
Users	id	full_name, email (UNIQUE), role	—
Categories	id	name (UNIQUE)	—
Statuses	id	name (UNIQUE)	—
Requests	id	title, created_at, category_id, status_id, applicant_id	category_id → Categories, status_id → Statuses, applicant_id → Users, executor_id → Users
Comments	id	request_id, user_id, text, created_at	request_id → Requests, user_id → Users

2.4. Нормализация.
Структура приведена к третьей нормальной форме: справочники Categories и Statuses вынесены в отдельные таблицы, в Requests хранятся только внешние ключи, дублирование названий категорий и статусов отсутствует.

2.5. SQL-скрипты.
Создан `schema.sql` — создание таблиц с нуля (с `DROP TABLE IF EXISTS` для повторного выполнения). Создан `seed.sql` — тестовые данные: 5 пользователей, 4 категории, 4 статуса, 5 заявок, 4 комментария.

2.6. Создание базы.
База SQLite создана скриптом `init_db.py`, который выполняет `schema.sql` и `seed.sql`. Файл базы: `docs/db/request_system.db`.

2.7. SQL-запросы.
Выполнены запросы: 5 INSERT, 5 SELECT, 2 UPDATE, 2 DELETE, 3 SELECT с WHERE, 2 JOIN. Результаты зафиксированы скриншотами. Файл с запросами: `docs/db/queries.sql`.

2.8. Работа с нейросетью.
Нейросеть использовалась для генерации ER-модели, чернового `schema.sql`, тестовых данных и сложных SQL-запросов. Результат проверен вручную: первичные и внешние ключи, типы данных, ограничения NOT NULL/UNIQUE, отсутствие дублирования, соответствие ER-диаграмме. Один SQL-фрагмент изменён самостоятельно (см. AI-журнал).

3. AI-журнал

Промпт 1 (ER-модель):
«Ты — проектировщик баз данных. Вот сущности системы учёта заявок: Users (пользователи с ролями), Requests (заявки), Statuses (статусы), Categories (категории), Comments (комментарии). Предложи ER-модель: таблицы, первичные и внешние ключи, ограничения NOT NULL/UNIQUE, связи 1:M. Формат: список таблиц с полями и типами + описание связей.»

Промпт 2 (SQL-запросы):
«Ты — SQL-разработчик. Для базы системы учёта заявок (таблицы Users, Requests, Statuses, Categories, Comments) предложи сложные SQL-запросы: JOIN, WHERE, агрегаты. Формат: запрос + пояснение, что он делает.»

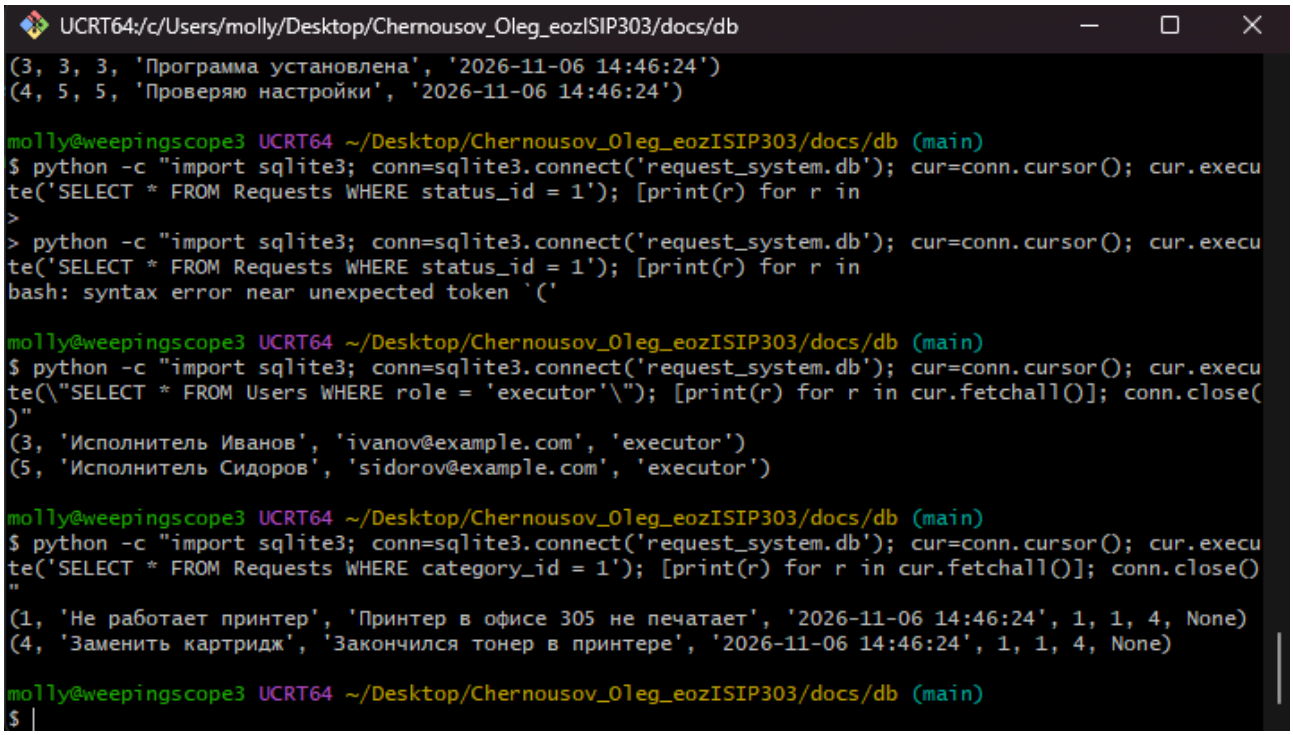
Запись AI-журнала:

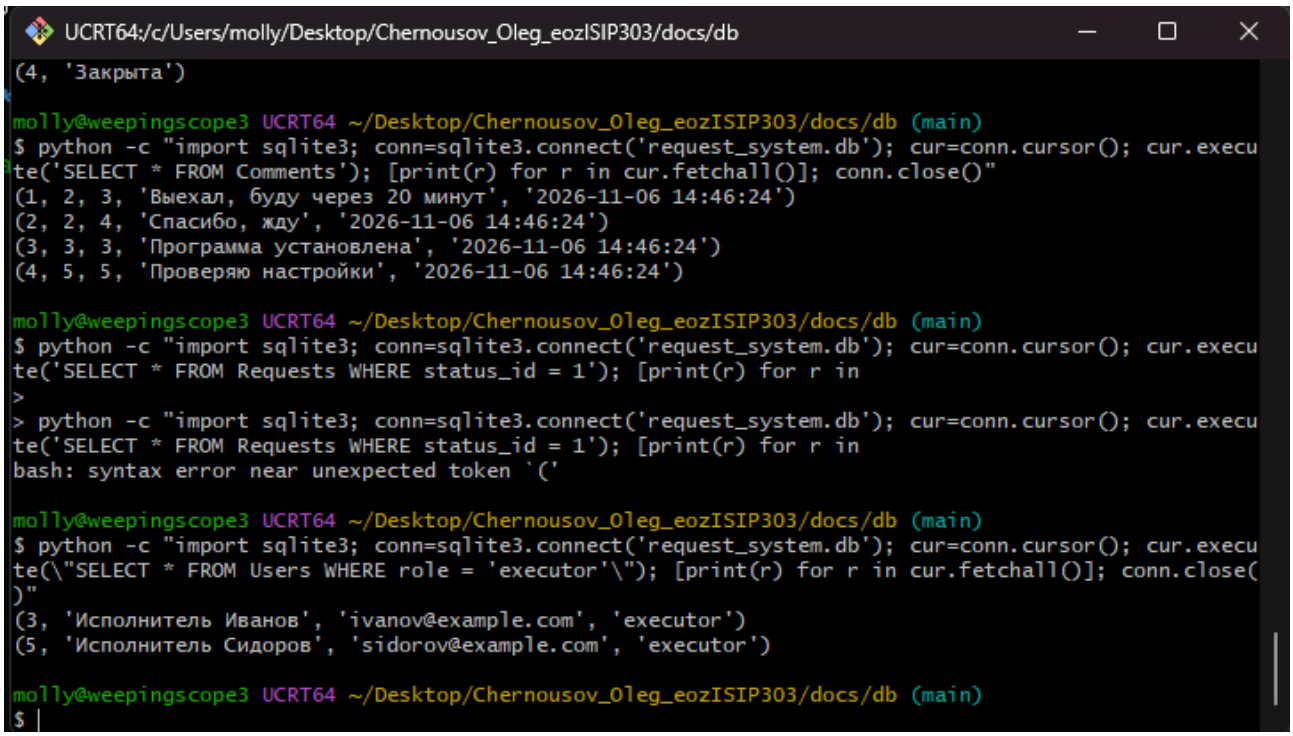
1. **Задача:** получить ER-модель, черновой `schema.sql`, тестовые данные и сложные SQL-запросы для системы учёта заявок.
2. **Модель:** DEEPSEEK-v3.
3. **Промпт:** Ты — проектировщик баз данных. Вот сущности системы учёта заявок: Users (пользователи с ролями), Requests (заявки), Statuses (статусы), Categories (категории), Comments (комментарии). Предложи ER-модель: таблицы, первичные и внешние ключи, ограничения NOT NULL/UNIQUE, связи 1:M. Формат: список таблиц с полями и типами + описание связей.
4. **Почему такой промпт:** задал роль (проектировщик БД, SQL-разработчик), контекст сущностей, формат (таблицы с полями, запросы с пояснениями) и ограничения — чтобы получить структурированный и проверяемый результат.
5. **Почему такая модель:** выбрана за качество работы с SQL и структурными ответами.
6. **Результат ИИ:** ER-модель с таблицами и ключами, черновой `schema.sql`, набор тестовых данных, сложные SQL-запросы с JOIN и WHERE.
7. **Проверка:** вручную проверил первичные и внешние ключи, типы данных, ограничения NOT NULL/UNIQUE, отсутствие дублирования; выполнил `schema.sql` и `seed.sql` в SQLite; выполнил все запросы и сверил результаты с тестовыми данными; сравнил ER-диаграмму со схемой.
8. **Собственная доработка:** изменил SQL-фрагмент из ответа ИИ — заменил `SELECT *` на явный список полей (`SELECT id, title, status_id`) в JOIN-запросе, потому что `SELECT *` неэффективен и неудобен для отчёта; добавил `DROP TABLE IF EXISTS` в начало `schema.sql`, чтобы скрипт можно было выполнять повторно; уточнил ограничение `CHECK` для роли пользователя.

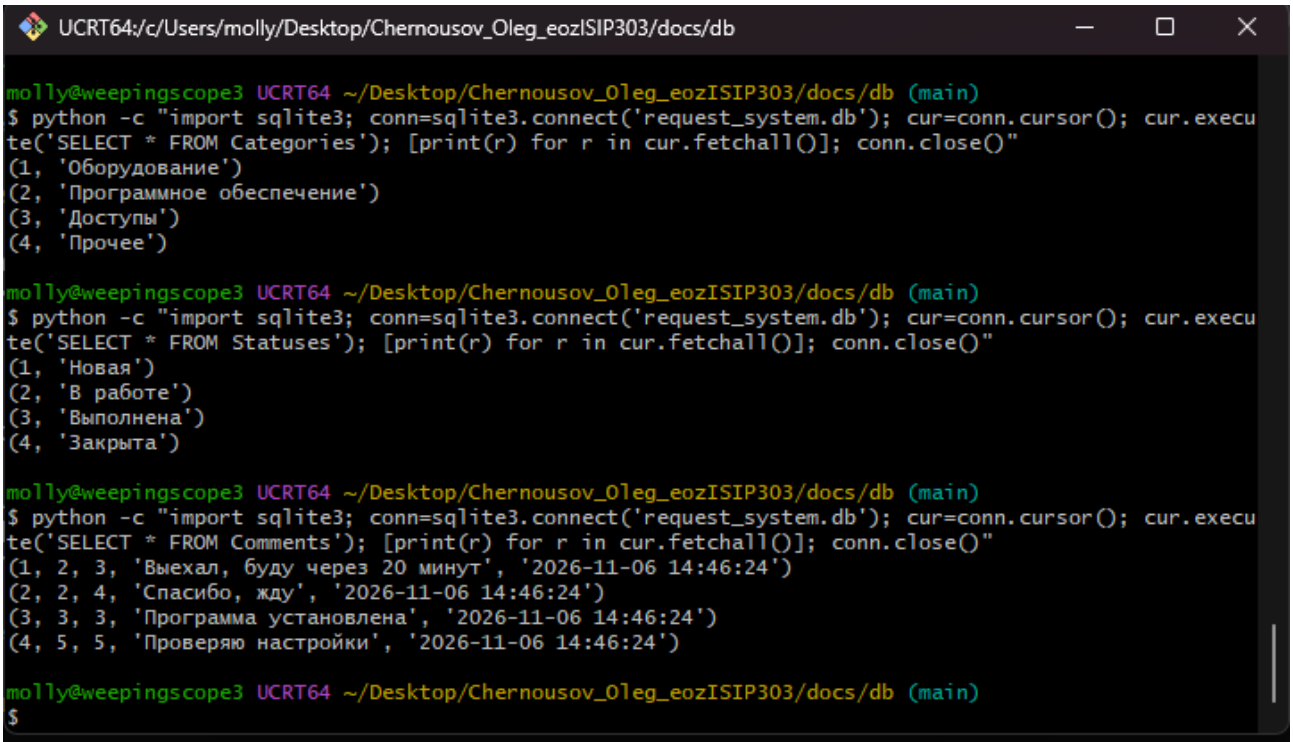
9. **Вывод:** ИИ хорошо генерирует черновую схему и запросы, но их обязательно нужно проверять на ключи, ограничения и соответствие ER-диаграмме; часть запросов пришлось доработать под реальные данные.

4. Вывод
В ходе работы освоен переход от модели предметной области к реляционной базе данных: выделены сущности и связи, построена ER-диаграмма, спроектированы 5 таблиц с первичными и внешними ключами, выполнена нормализация. Созданы SQL-скрипты `schema.sql` и `seed.sql`, база SQLite, выполнены запросы (INSERT, SELECT, UPDATE, DELETE, WHERE, JOIN) с фиксацией результатов. Нейросеть использовалась как инструмент генерации черновиков; результат проверен и доработан самостоятельно. Все файлы размещены в репозитории.

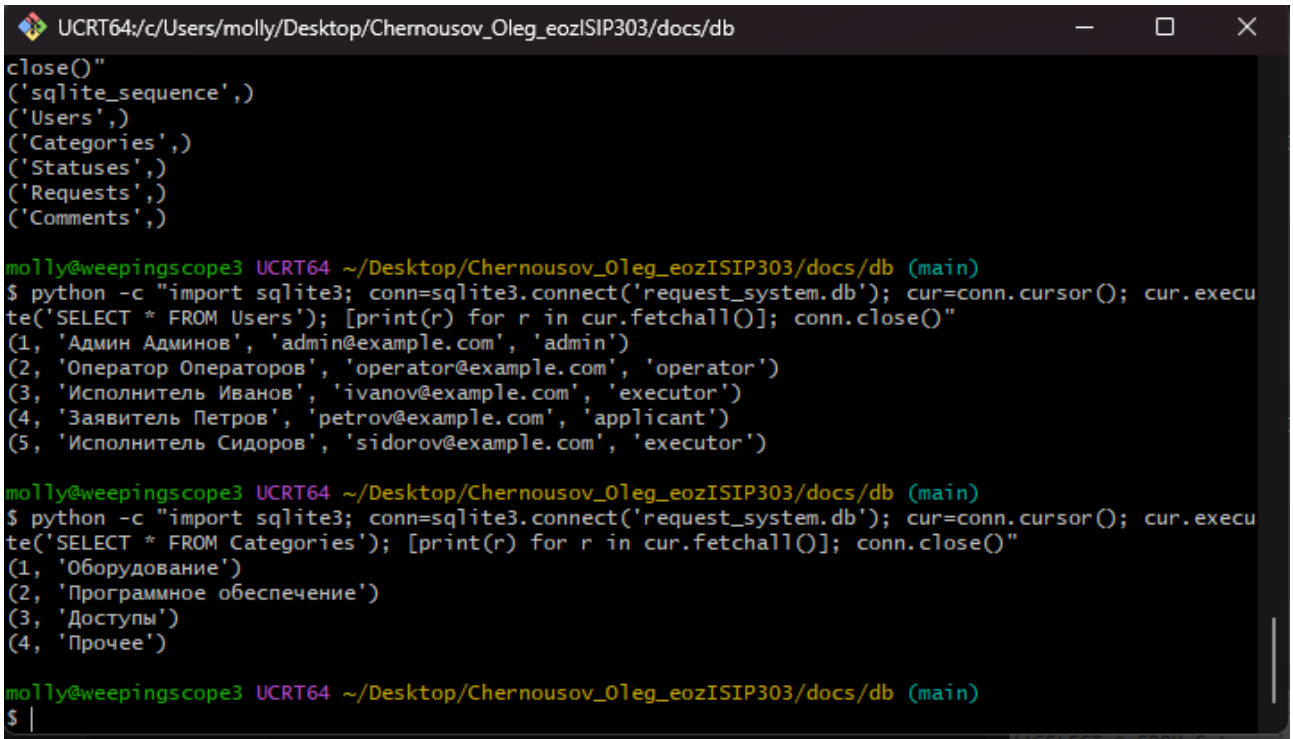
Скриншоты работоспособности




UCRT64:/c/Users/molly/Desktop/Chernousov_Oleg_eozISIP303/docs/db
(4, 'Закрыта')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Comments'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 2, 3, 'Выехал, буду через 20 минут', '2026-11-06 14:46:24')
(2, 2, 4, 'Спасибо, жду', '2026-11-06 14:46:24')
(3, 3, 3, 'Программа установлена', '2026-11-06 14:46:24')
(4, 5, 5, 'Проверяю настройки', '2026-11-06 14:46:24')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Requests WHERE status_id = 1'); [print(r) for r in
>
> python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Requests WHERE status_id = 1'); [print(r) for r in
bash: syntax error near unexpected token `('
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te(\"SELECT * FROM Users WHERE role = 'executor'\"); [print(r) for r in cur.fetchall()]; conn.close(
)"
(3, 'Исполнитель Иванов', 'ivanov@example.com', 'executor')
(5, 'Исполнитель Сидоров', 'sidorov@example.com', 'executor')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$


UCRT64:/c/Users/molly/Desktop/Chernousov_Oleg_eozISIP303/docs/db
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Categories'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Оборудование')
(2, 'Программное обеспечение')
(3, 'Доступы')
(4, 'Прочее')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Statuses'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Новая')
(2, 'В работе')
(3, 'Выполнена')
(4, 'Закрыта')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Comments'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 2, 3, 'Выехал, буду через 20 минут', '2026-11-06 14:46:24')
(2, 2, 4, 'Спасибо, жду', '2026-11-06 14:46:24')
(3, 3, 3, 'Программа установлена', '2026-11-06 14:46:24')
(4, 5, 5, 'Проверяю настройки', '2026-11-06 14:46:24')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$


UCRT64:/c/Users/molly/Desktop/Chernousov_Oleg_eozISIP303/docs/db
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Users'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Admin Adminov', 'admin@example.com', 'admin')
(2, 'Operator Operatorov', 'operator@example.com', 'operator')
(3, 'Ispolnitel Ivanov', 'ivanov@example.com', 'executor')
(4, 'Zayavitel Petrov', 'petrov@example.com', 'applicant')
(5, 'Ispolnitel Sidorov', 'sidorov@example.com', 'executor')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Categories'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Oborudovanie')
(2, 'Programmnoe obespechenie')
(3, 'Dostupy')
(4, 'Prochee')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Statuses'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Novaya')
(2, 'V rabote')
(3, 'Vypolnena')
(4, 'Zakryta')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$


UCRT64:/c/Users/molly/Desktop/Chernousov_Oleg_eozISIP303/docs/db
close()"
('sqlite_sequence',)
('Users',)
('Categories',)
('Statuses',)
('Requests',)
('Comments',)
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Users'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Admin Adminov', 'admin@example.com', 'admin')
(2, 'Operator Operatorov', 'operator@example.com', 'operator')
(3, 'Ispolnitel Ivanov', 'ivanov@example.com', 'executor')
(4, 'Zayavitel Petrov', 'petrov@example.com', 'applicant')
(5, 'Ispolnitel Sidorov', 'sidorov@example.com', 'executor')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$ python -c "import sqlite3; conn=sqlite3.connect('request_system.db'); cur=conn.cursor(); cur.execu
te('SELECT * FROM Categories'); [print(r) for r in cur.fetchall()]; conn.close()"
(1, 'Oborudovanie')
(2, 'Programmnoe obespechenie')
(3, 'Dostupy')
(4, 'Prochee')
molly@weepingscope3 UCRT64 ~/Desktop/Chernousov_Oleg_eozISIP303/docs/db (main)
$